

L15: Reneging and Customer Impatience

Simulation Without a Black Box

Outline

Learning Objectives

Reneging: Definition

SimPy Timeout-Race Pattern

State Update on Renege

Balking

Combined Model

Performance Measures

Code Walkthrough Summary

Key Takeaways

Learning Objectives

By the end of this lecture you will be able to:

- Define reneging and balking and distinguish them from blocking.
- Implement the SimPy timeout-race pattern to model reneging.
- Update server and queue state correctly when a customer reneges.
- Combine reneging with a finite buffer in a single model.
- Compute abandonment rate and conditional wait (given service is received).

Reneging: Customers Abandon After Waiting Too Long

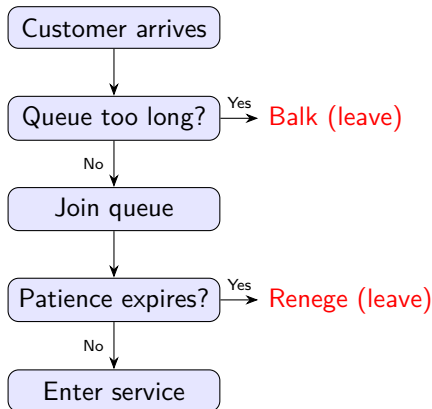
Definitions

Reneging Customer *joins* the queue but *leaves* after waiting longer than their patience T_p .

Balking Customer *does not join* upon arrival (e.g., queue length too long).

Blocking Customer *cannot join* — system is at capacity (M/M/1/K).

Typical patience distributions: exponential, uniform, or deterministic threshold.



SimPy Timeout-Race: `yield req | env.timeout(patience)`

```
import simpy, numpy as np
```

```
def customer(env, server, rng, stats):
    arrive = env.now
    patience = rng.exponential(5.0)    # mean patience = 5 time units
    req = server.request()

    result = yield req | env.timeout(patience)

    if req in result:                  # server granted before timeout
        wait = env.now - arrive
        stats["served_waits"].append(wait)
        yield env.timeout(rng.exponential(1.0))  # service
        req.release(req)
    else:                             # patience expired first
        req.cancel()                   # remove from server's queue
        stats["renege"] += 1
        stats["renege_waits"].append(env.now - arrive)
```

Key

`yield A | B` returns as soon as *either* event fires. Test which one won with `if req in result`

State Update When a Customer Reneges

```
# After reneging:  
req.cancel()           # critical: remove request from server.queue  
                       # otherwise the server stays "reserved" forever  
stats["reneged"] += 1
```

Common mistake

Forgetting `req.cancel()` after reneging leaves a ghost request in the resource queue. The next customer then waits forever for a server that thinks it is still busy.

- After `req.cancel()`, SimPy removes the request from `server.queue`.
- The server's count is unchanged (the reneging customer never held the resource).
- Log both `env.now` (renege time) and the original arrival time to compute sojourn.

Balking: Don't Join a Long Queue

```
MAX_VISIBLE = 4    # customer balks if  $\geq 4$  already waiting
```

```
def customer(env, server, rng, stats):  
    if len(server.queue)  $\geq$  MAX_VISIBLE:  
        stats["balked"] += 1  
        return        # never joins  
  
    arrive = env.now  
    with server.request() as req:  
        yield req  
        stats["served_waits"].append(env.now - arrive)  
        yield env.timeout(rng.exponential(1.0))
```

- Balking is checked **before** the `with server.request()` block.
- `len(server.queue)` counts customers *waiting* (not in service).
- For probabilistic balking: `if rng.random() < p_balk(len(server.queue)):`
 `return.`

Combined Model: Reneging + Finite Buffer

```
def customer(env, server, K, rng, stats):  
    # 1. Balking check (finite buffer)  
    if server.count + len(server.queue) >= K:  
        stats["blocked"] += 1  
        return  
  
    arrive = env.now  
    patience = rng.exponential(5.0)  
    req = server.request()  
    result = yield req | env.timeout(patience)  
  
    if req in result:  
        stats["served_waits"].append(env.now - arrive)  
        yield env.timeout(rng.exponential(1.0))  
        req.release(req)  
    else:  
        req.cancel()  
        stats["reneged"] += 1
```


Performance Measures for Impatient Queues

Why conditional wait matters

Measure	Formula / source	$E[W \text{served}]$ understates true congestion because renegers <i>left early</i> . Report both:
Abandonment rate	reneged/arrived	$E[W \text{served}]$: perceived wait by satisfied customers. - Abandonment rate: fraction of demand lost.
Blocking rate	blocked/attempted	
Service fraction	served/arrived	
Eff. throughput	served/ T	
Cond. wait $E[W \text{served}]$	mean(served_waits)	
Cond. wait $E[W \text{reneged}]$	mean(renege_waits)	

Together they give a complete picture of system performance.

Reneging Pattern: Code Walkthrough Summary

1. **Create** the request before the yield: `req = server.request()`.
2. **Race** the request against a patience timeout: `yield req | env.timeout(p)`.
3. **Test** the result dictionary: `if req in result` → served, else → reneged.
4. If reneged: **cancel** the request with `req.cancel()` and log the abandonment.
5. If served: proceed with `env.timeout(service_time)` then **release** via `req.release(req)`.

Do not use `with server.request()` as `req`: for reneging

The `with` block auto-releases on exit — you cannot cancel a request inside it after a timeout race. Manage the request manually.

Key Takeaways

Core ideas from L15

1. **Reneging** = customer abandons from queue; **balking** = customer never joins; **blocking** = system full.
2. The SimPy **timeout-race** (`yield req | env.timeout(p)`) is the standard pattern for impatience.
3. Always call `req.cancel()` after reneging — forgetting it corrupts the resource state.
4. Balking is a simple pre-arrival check on `len(server.queue)`.
5. Report **abandonment rate** alongside $E[W|\text{served}]$ for a complete picture.

Next: L16 — Priority queues and preemption in SimPy.